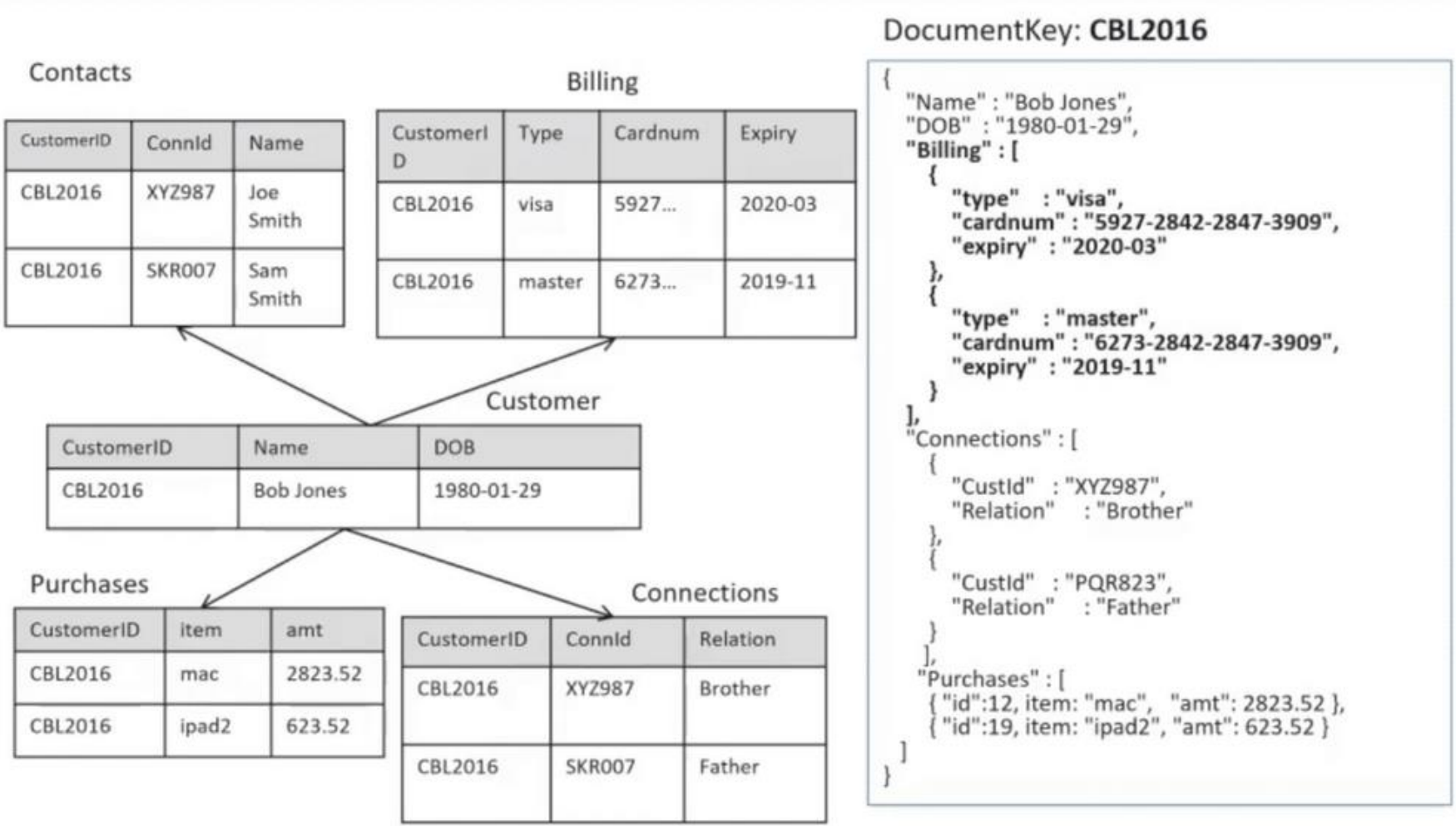
NoSQL – Document Databases

Course 6

Document Databases

Are known for their flexibility, and this flexibility primarily stems from their ability to store data in a schemaless or semi-structured format:

- **Schemaless Design:** a rigid, predefined schema is not enforced. They allow data to be stored in a flexible, document-oriented format, typically using formats such as JSON or BSON (Binary JSON).
- **Dynamic Schema Evolution:** as application requirements evolve, the model can easily adapt without the need for complex migrations. New fields can be added to documents, and existing fields can be modified or eliminated as needed.
- **Nested and Complex Data Structures:** they support nested and complex data structures within documents. As example, a document can contain arrays, sub-documents, or other documents, allowing the representation of complex, nested data.



Source: Couchbase

Document Databases

- **Polymorphic Data**: documents within a collection can represent different types of entities or objects with different attributes. This allows for storing different data types in the same collection.
- **Improved Read and Write Performance**: because associated data is often stored together within a single document, this can provide improved read and write performance for certain types of queries. Selecting a complete document is often more efficient than performing joins.
- **Query Flexibility**: they support flexible and expressive query languages that allow querying based on the content of the documents.
- **Aggregation Frameworks**: they offer aggregation frameworks that enable complex data processing and transformation within the database. This allows for the execution of advanced analysis and data manipulation operations directly on the stored documents.
- **Horizontal Scalability**: they are designed to scale horizontally by distributing data across multiple nodes or servers.

Document Databases

The flexibility offered lies in their ability to adapt to dynamic data models, handle diverse and nested data structures, and provide efficient querying capabilities. This flexibility makes document-oriented databases a suitable choice for applications with changing requirements and complex data structures.

Key-Value Pairs vs. JSON/BSON Documents

- Key-Value Pairs:

Structure: In a key-value pair data structure, each piece of data is associated with a unique identifier known as the key. The key is used to retrieve the corresponding value.

Use Cases: Key-value collections are often simple and efficient, making them suitable for scenarios where fast and direct data access is crucial. These databases are commonly used for caching and metadata storage.

Document Databases

- JSON/BSON Documents

  Structure:

  JSON (JavaScript Object Notation): A data exchange format that uses an easily understandable and readable text format. It supports data types such as objects, arrays, strings, numbers, boolean values, and null.

  BSON (Binary JSON): A binary-encoded serialization of JSON-like documents. BSON extends JSON to include additional data types, such as binary data and the date type, making it more efficient for storage and data exchange.

  Use Cases:

  JSON/BSON documents are the primary data structure in document-oriented databases. Each document is a self-contained unit that represents a set of key-value pairs, where the keys are field names and the values can be simple data types, arrays, nested documents, or a combination thereof.

JSON vs BSON

Feature	JSON (JavaScript Object Notation)	BSON (Binary JSON)
Format	**Text-based**	**Binary-encoded**
Readability	**High**. Easily readable and editable by humans with any text editor.	**Low**. Not human-readable without a specific decoder/tool.
Parsing Speed	**Slower**. The text needs to be parsed into data types (e.g., a date string must be converted to a `Date` object).	**Faster**. Designed for machine processing, allowing quicker serialization and deserialization by the database engine.
Interchange	**Excellent** for APIs and data exchange between different systems due to its ubiquity.	Primarily used **internally** by document databases (like MongoDB) for storage and retrieval.

JSON vs BSON

Data Type	JSON Support	BSON Support (Example Types)
Numeric	Single generic **number** type (float/double).	Separate types for **int32**, **int64**, **double**, and **Decimal128**.
Date/Time	Stored as a **string** (e.g., "2025-11-02T...Z").	Stores a native **Date** type (a 64-bit integer representing milliseconds).
Binary Data	No native type; must be encoded as a **base64 string**.	Has a native **Binary** type.
Identifiers	No native type.	Supports **ObjectId** (used by MongoDB for its default _id field).

JSON vs BSON

Aspect	JSON	BSON
Storage Size	Typically **smaller** for simple data because it lacks the BSON metadata overhead.	**Variable**. Can be **larger** for small documents due to metadata, but often **more efficient** for large, complex, or binary data.
Traversal	Slower, as it lacks a fixed length, meaning the parser must read the entire document to find a nested field.	**Faster**. It encodes **type and length information** for each field. This allows the database to "skip" over parts of the document it doesn't need to read when performing a query.
Indexing	Limited to indexing the string representation of data.	**Powerful**. The database can index and query based on the native BSON types (like a native `Date` object).

BSON Architecture

structured as a series of ordered field-value pairs encoded in a binary format. This structure is designed for fast traversal, scanning, and indexing.
The basic architecture of a BSON document is:

1. Total Document Size: the document begins with a 32-bit integer that specifies the total size of the document in bytes. This allows a parser to skip the entire document quickly.
2. Element List (e_list): a sequence of elements, each representing a field-value pair. The field-value pairs are stored with a defined order.
    - Element Structure: Each element is composed of:
        - Type: A single byte that explicitly indicates the BSON data type (e.g., 0x02 for String, 0x12 for UTC Datetime).
        - Field Name: A C-string (UTF-8 encoded string terminated by a null byte) that serves as the key.
        - Value: The actual data, encoded according to the specified type. For compound types like nested documents or arrays, the value includes its own size prefix.
3. Terminating Null Byte: the document ends with a single null byte (0x00) to mark the end of the element list.

JSON vs BSON

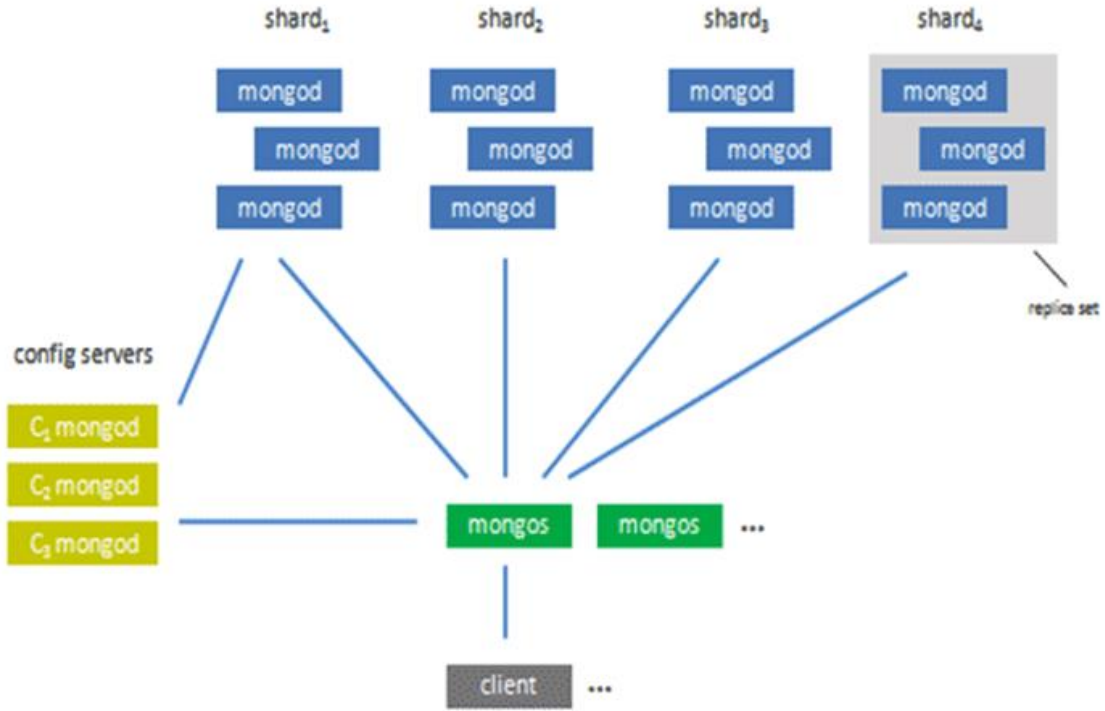
```
{
  "name": "Alice",
  "age": 30,
  "active": true
}
```

Field/Component	BSON Code (Hex) / Value	Description	Purpose/JSON Equivalent
Total Size	1D 00 00 00 (29 in decimal)	4-byte **integer** specifying the **total length** of the document (including this field itself and the terminator).	Allows fast skipping/parsing.
Elem 1: "name" (String)			
Type	2	**type byte** - the next value is a **String**.	name is a string.
Field Name	6E 61 6D 65 00 ("name\0")	The key, followed by a **null terminator**.	"name"
String Size	06 00 00 00 (6 in decimal)	A 4-byte integer specifying the length of the string value **plus its null terminator**.	The string value is 5 characters.
Value	41 6C 69 63 65 00 ("Alice\0")	The UTF-8 string value, also **null-terminated**.	"Alice"
Elem 2: "age" (Integer)			
Type	10	**type byte** - the next value is a **32-bit Integer**.	age is a number.
Field Name	61 67 65 00 ("age\0")	The key, followed by a **null terminator**.	"age"
Value	1E 00 00 00 (30 in little-endian)	The 4-byte binary integer value.	30
Elem 3: "active" (Boolean)			
Type	8	**type byte** - the next value is a **Boolean**.	active is a boolean.
Field Name	69 73 5F 61 63 74 69 76 65 00 ("active\0")	the key, followed by a **null terminator**.	"active"
Value	1	single byte: 01 - true, 00 - false.	TRUE
Document Terminator	0	final single **null byte**.	end of JSON object.

MongoDB

- Is a cross-platform, document-oriented database that offers high performance, high availability, and easy scalability.
- It uses the concept of a collection and a document. The database is a physical container for collections. Each database has its own set of files on the file system. A single MongoDB server typically has multiple databases.
- A **collection** is a group of MongoDB documents (the equivalent of a table). A collection can belong to only one database. No schema is enforced.
- **Documents** within a collection can have different fields. A document is a set of key-value pairs.

MongoDB Architecture



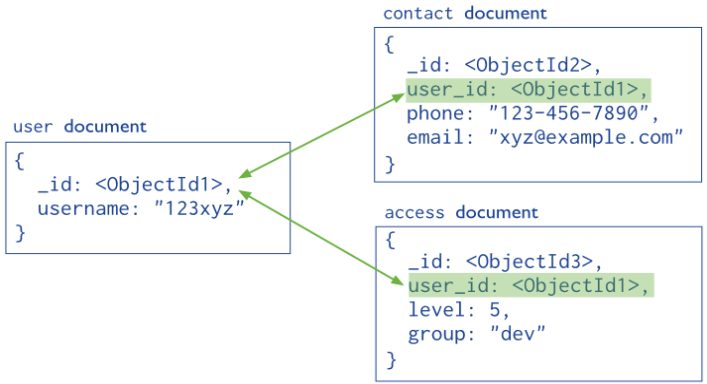
source: mongodb.com

Documents

An important decision in designing data models is the structure of the documents and how the application represents relationships between the data. There are two ways to represent these relationships:

- **References**: Store relationships between data by including links or references from one document to another.
- **Nested** Documents (or **Embedded** Documents): Store related data within a single document structure. MongoDB documents make it possible to embed document structures within a field or an array within a document.

Documents





source: mongodb.com

Relationships

1 to 1 with nesting

```
{
    patron_id: "joe",
    street: "123 Fake Street",
    city: "Faketon",
    state: "MA",
    zip: "12345"
}
```

```
{
    _id: "joe",
    name: "Joe Bookreader",
    address: {
                street: "123 Fake Street",
                city: "Faketon",
                state: "MA",
                zip: "12345"
             }
}
```

Relationships

1 to N with nesting

```
{
    _id: "joe",
    name: "Joe Bookreader"
}

{
    patron_id: "joe",
    street: "123 Fake Street",
    city: "Faketon",
    state: "MA",
    zip: "12345"
}

{
    patron_id: "joe",
    street: "1 Some Other Street",
    city: "Boston",
    state: "MA",
    zip: "12345"
}
```



```
{
    _id: "joe",
    name: "Joe Bookreader",
    addresses: [
                {
                    street: "123 Fake Street",
                    city: "Faketon",
                    state: "MA",
                    zip: "12345"
                },
                {
                    street: "1 Some Other Street",
                    city: "Boston",
                    state: "MA",
                    zip: "12345"
                }
            ]
}
```

Relationships

1 to N
with references

```
{
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}

{
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}
```



```
{
   name: "O'Reilly Media",
   founded: 1980,
   location: "CA",
   books: [12346789, 234567890, ...]
}

{
    _id: 123456789,
    title: "MongoDB: The Definitive Guide",
    author: [ "Kristina Chodorow", "Mike Dirolf" ],
    published_date: ISODate("2010-09-24"),
    pages: 216,
    language: "English"
}

{
   _id: 234567890,
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English"
}
```

Relationships

1 to N with references

```
{
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}

{
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}
```



```
{
   _id: "oreilly",
   name: "O'Reilly Media",
   founded: 1980,
   location: "CA"
}

{
   _id: 123456789,
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher_id: "oreilly"
}

{
   _id: 234567890,
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher_id: "oreilly"
}
```

Documents

Data is stored in documents: JSON-style data structures composed of key and value pairs. On disk, the documents are stored in the BSON serialization format. BSON is a binary representation of JSON documents, although it contains more data types than JSON.

```
var mydoc = {
               _id: ObjectId("5099803df3f4948bd2f98391"),
               name: { first: "Alan", last: "Turing" },
               birth: new Date('Jun 23, 1912'),
               death: new Date('Jun 07, 1954'),
               contribs: [ "Turing machine", "Turing test", "Turingery" ],
               views : NumberLong(1250000)
             }
```

_id

is the primary key for every document in a MongoDB collection. Its role is analogous to a primary key column in a relational database table, but it comes with some unique characteristics essential to how MongoDB operates.
Characteristics:

- Uniqueness: Every document within a collection must have a unique _id field.
- Immutability: The value of the _id field cannot be changed once the document is created.
- Indexing: MongoDB automatically creates a unique index on the _id field for every collection. This ensures fast lookups and efficient enforcement of uniqueness.

_id

Default _id Value: The ObjectId
When insert a document without explicitly providing an _id field, MongoDB automatically generates a value using the ObjectId data type.
Structure of an ObjectId
Is a 12-byte BSON value that is typically represented as a 24-character hexadecimal string. It's designed to be unique, lightweight, and ordered. It is composed of four sections:

4-byte Timestamp: Represents the seconds since the Unix epoch (the document's creation time). This is why ObjectIds are naturally ordered by time.

5-byte Random Value: A machine identifier and process ID combination.

3-byte Incrementing Counter: A counter that starts at a random value.

Because the timestamp is the first 4 bytes, sorting documents by their _id provides a rough chronological ordering, which is often very useful for time-series data or activity feeds.

_id

While the ObjectId is the default and often the best choice, it is not required to use. It can be explicitly set the _id to a value of any valid BSON data type other than an array:

Data Type	Example Use Case
String	Usernames ("john.doe"), email addresses, or custom sequential IDs (e.g., product SKUs).
Integer	Sequential numbers from an external system.
BSON Date	Useful when the date itself is the primary unique identifier for a document.

```
{
  "_id": "user@example.com",
  "name": "User Example",
  " status": "Active“
}
```

MongoDB Index Types

MongoDB supports several index types:

1. Single-Field and Compound Indexes

   Single-Field: An index on a single field in your document (e.g., { email: 1 }).

   Compound: An index on multiple fields (e.g., { country: 1, city: -1 }). The order of the fields is crucial, following the ESR Rule (Equality, Sort, Range) for optimal query efficiency.

2. Multikey Indexes (Arrays)

   If a field in your document is an array and you create an index on it, MongoDB automatically creates a Multikey Index.

   It creates a separate index entry for every element in the array.

   Use Case: Searching documents based on items within a list, such as a document with a tags array.

MongoDB Index Types

3. Geospatial Indexes (2dsphere & 2d)
   - 2dsphere: Used to index and query geospatial data defined on an Earth-like sphere. It supports queries like find documents within a polygon or find the nearest locations.
   - 2d: Used for planar (flat) geometry or simple Cartesian coordinates.
4. Text Indexes
   - Supports full-text search queries on string content in your collections.
   - The index stores all the significant words from the indexed fields and enables linguistic-aware searches.
   - Use Case: Searching a blog post or product catalog for keywords.
5. Hashed Indexes
   - These indexes store a hash of the value of the indexed field.
   - They are primarily used to support sharding based on a hashed shard key.
   - Limitation: Only support equality matches, not range queries.

MongoDB Index Types

Index Type	Example Document	MongoDB Shell Command
Single-Field	{ username: "user1" }	db.users.createIndex({ username: 1 })
Compound	{ country: "USA", city: "NYC" }	db.stores.createIndex({ country: 1, city: -1 })
Unique	{ email: "a@b.com" }	db.users.createIndex({ email: 1 }, { unique: true })
Multikey (Array)	{ tags: ["a", "b"] }	db.products.createIndex({ tags: 1 })
Geospatial	{ loc: [40, 74] }	db.restaurants.createIndex({ loc: "2dsphere" })
Text	{ description: "..." }	db.articles.createIndex({ description: "text" })

MongoDB Index Types

Option	Description	Use Case
unique: true	Prevents the collection from having documents with duplicate values for the indexed key.	Enforcing uniqueness for fields like email or SSN.
sparse: true	Only indexes documents that contain the indexed field. Documents missing the field will **not** be indexed.	When an optional field only needs to be indexed for a subset of documents.
expireAfterSeconds	Allows MongoDB to automatically remove documents after a certain time period. These are called **TTL (Time To Live) Indexes**.	Automatic cleanup of session data, logs, or cache documents.
background: true	Builds the index in the background, minimizing the impact on database operations. (Default is often foreground for faster creation).	Large production collections where downtime is unacceptable.

Queries - Read

Queries in MongoDB are performed using the MongoDB Query Language (MQL) and involve specifying conditions to retrieve documents from collections based on certain criteria. MongoDB provides a rich set of query operators and methods for filtering, sorting, and projecting data.
Read operations include all operations that return a cursor in response to queries, and additionally include a number of aggregation operations that do not return a cursor but have properties similar to queries. These commands include `aggregate`, `count` and `distinct`.
In the Mongo shell, the `find()` and `findOne()` methods are used for read operations.

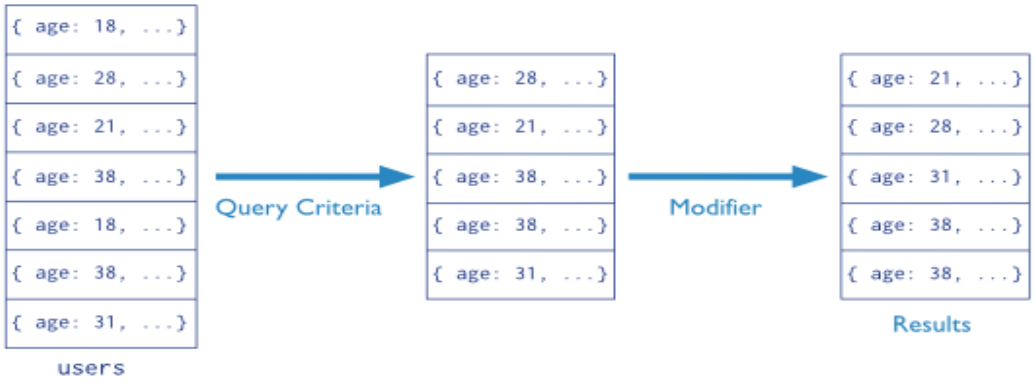
Queries - Read

```
db.users.find(
    { age: { $gt: 18 } },
    { name: 1, address: 1 }
).limit(5)
```

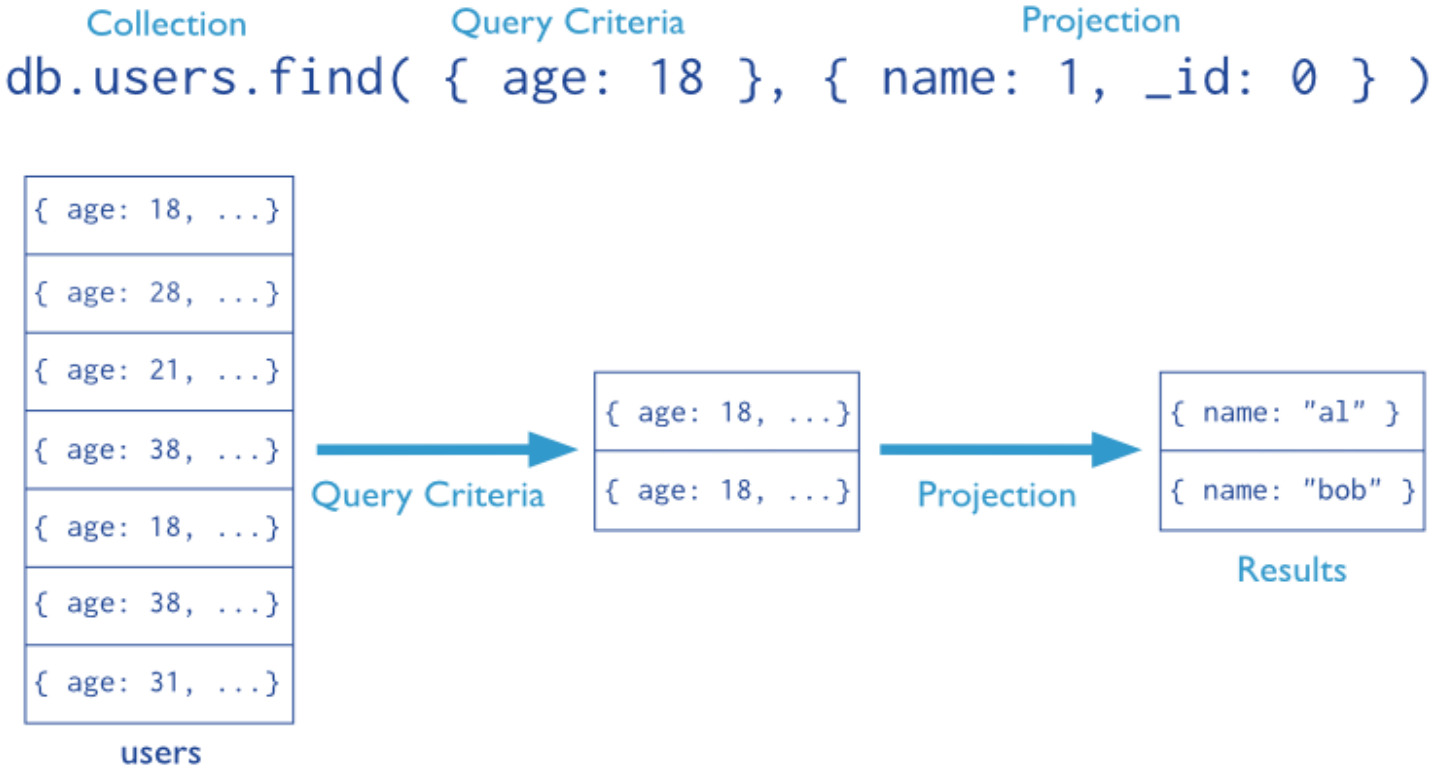
- collection
- query criteria
- projection
- cursor modifier

Collection | Query Criteria | Modifier

```
db.users.find( { age: { $gt: 18 } } ).sort( {age: 1 } )
```



Queries - Read



Queries - Write

Write operations are atomic (at the document level);
A write operation can atomically affect only a single document and a single collection.
Operations that modify more than one document in a collection act on only one document at a time.
A denormalized data model with embedded data facilitates atomic write operations, since a single write operation can insert or update the data for an entity.
In a normalized model, the data would be split across multiple collections and would require multiple write operations that are not collectively atomic.

Queries - Write

- Insert: adding new data into collection

```
db.users.insert (          <- collection
   {
     name: "sue",          <- field: value
     age: 26,              <- field: value
     status: "A"           <- field: value
   }                       } document
)
```

- Update: modify data (criteria can be used)

```
db.users.update(                    <- collection
   { age: { $gt: 18 } },            <- update criteria
   { $set: { status: "A" } },       <- update action
   { multi: true }                  <- update option
)
```

- Remove: delete data (criteria can be used)

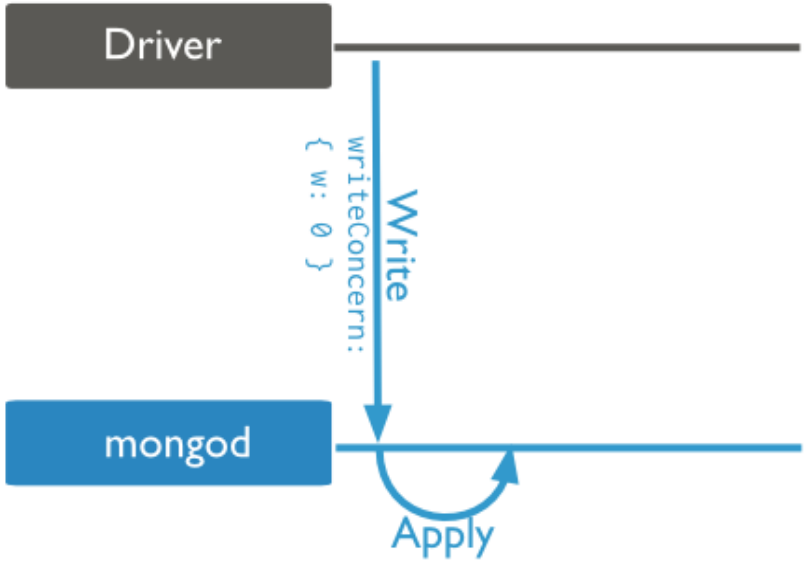
```
db.users.remove(          <- collection
   { status: "D" }        <- remove criteria
)
```

Write Concern

The guarantee provided when reporting the successful execution of a write operation.
The "write concern" configuration determines the level of this guarantee (similar to the isolation level).
Write operations with a weak write concern execute quickly. In some failure cases, write operations initiated with a weak write concern may not be persistent.
With a stronger write concern, the client application must wait longer for the confirmation of write operations.
MongoDB offers different levels of write concern to suit various needs. The write concern can be adjusted to balance performance and persistence requirements.
By default, it is set to "Acknowledged."

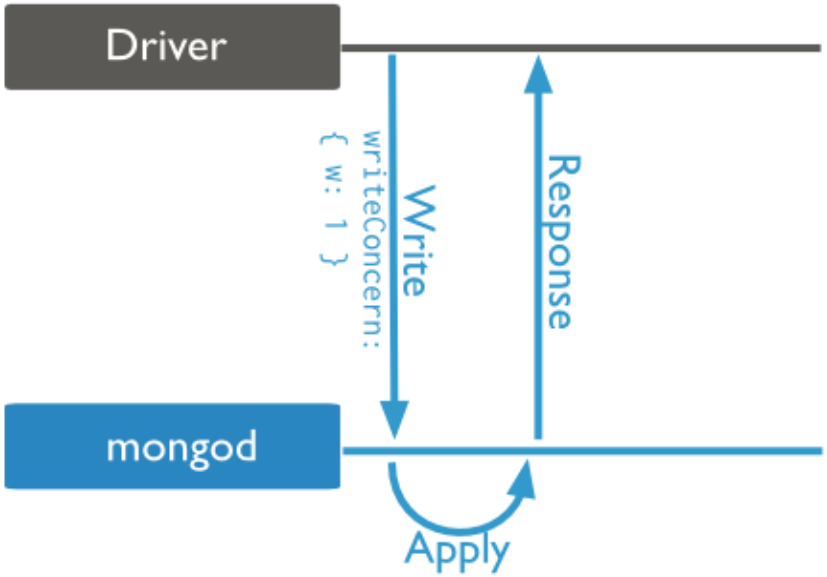
Write Concern - Unacknowledged

Using this mode, MongoDB does not send a confirmation for the write operation.



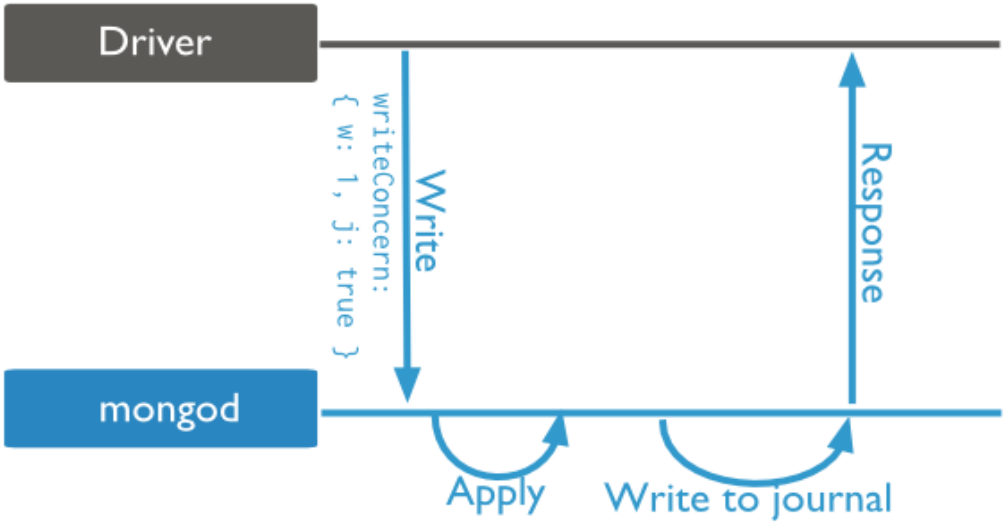
Write Concern - Acknowledged

Using this mode, MongoDB sends a confirmation for the receipt of the write operation request and data saving in memory.



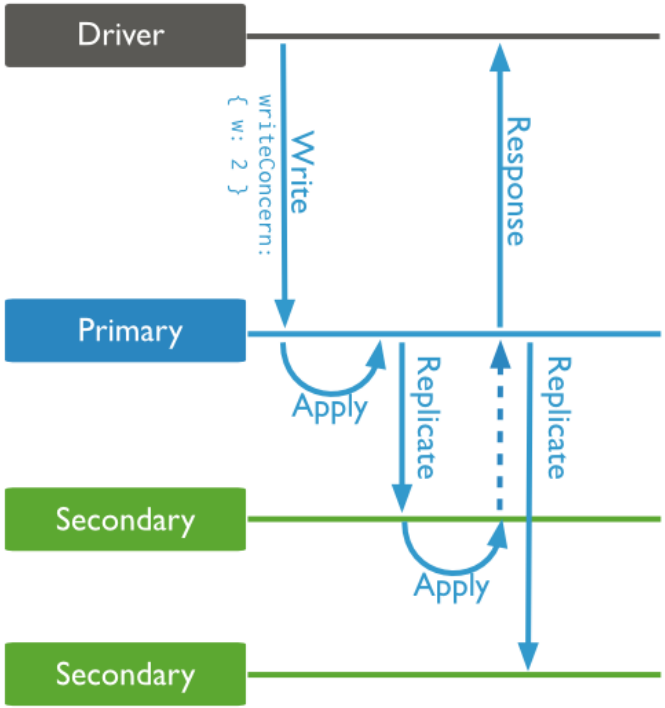
Write Concern - Journaled

In this mode, MongoDB acknowledges the receipt of the write request only after the write operation has been saved to the log file. This ensures that the state can be restored after a shutdown or power failure.



Write Concern – Replica Acknowledged

In this mode, MongoDB guarantees that the write operation is propagated to all components of a replica set.



Aggregations

Aggregation operations process multiple documents and return computed results. They are useful for:

- Grouping values from multiple documents.
- Performing operations on grouped data to return a single result.
- Analyzing data changes over time.

For aggregation operations, you can use:

- Aggregation pipelines (the recommended method).
- Single-purpose aggregation methods (they are simple but lack the capabilities of a pipeline).

Aggregations – Pipes

An aggregation pipeline consists of one or more stages for processing documents:

- Each stage performs an operation on the input documents, such as filtering, grouping, or calculating values.
- The documents generated in one stage are passed to the next stage.
- An aggregation pipeline can return results for groups of documents, such as totals, averages, maximums, or minimums.
- Operations can be performed on collections stored in shards.
- There are limitations on the result size, the number of stages, and the amount of memory used.

Aggregations – Pipes

```
db.orders.aggregate( [

   // Stage 1: Filter pizza order documents by pizza size
   {
      $match: { size: "medium" }
   },

   // Stage 2: Group remaining documents by pizza name and calculate total quantit
   {
      $group: { _id: "$name", totalQuantity: { $sum: "$quantity" } }
   }

] )
```

Aggregations – Uniscope Functions

Single-purpose aggregation methods (they are simple but lack the capabilities of a pipeline):

- `db.collection.estimateDocumentCount()`: Provides an approximate total number of documents in a collection.
- `db.collection.count()`: Provides the number of documents in a collection.
- `db.collection.distinct()`: Returns an array of documents that have distinct values for the specified field.

Map - Reduce

It is a paradigm for processing large volumes of data and generating aggregations.

The `map()` function is applied to each input document and emits key-value pairs.

In the `reduce()` stage, multiple values for the same key are aggregated.

